



Jeu du Taquin 3×3

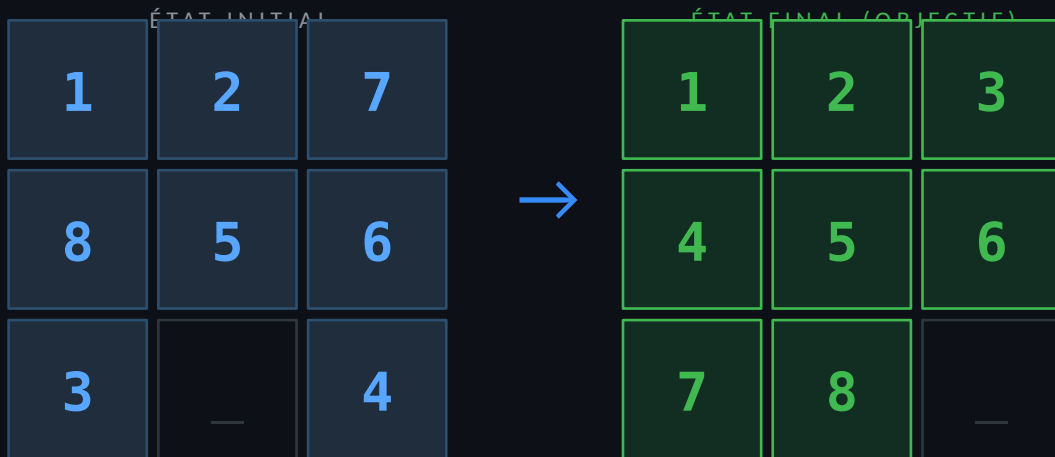
Rapport d'analyse

Mini Projet IA • Fondements de l'Intelligence Artificielle
Institut Supérieur d'Informatique • ISI 2025-2026
Rayen Bouyahia - L2CS01

1

Présentation du problème

Le jeu du taquin 3×3 consiste à remettre dans l'ordre 8 tuiles numérotées sur une grille 3×3, en glissant les tuiles vers la case vide adjacente. L'espace d'états compte au maximum $9! / 2 = 181\,440$ états atteignables.

**8**

Tuiles

3×3

Grille

181 440

États max

23 mvts

Solution optimale

Algorithmes implémentés

DFS**Depth-First Search**

✗ Non optimal

Structure : Pile LIFO (`list.pop()`)

Plonge en profondeur avant de revenir. Peu coûteux en mémoire mais ne garantit pas le chemin optimal.

BFS**Breadth-First Search**

✓ Optimal

Structure : File FIFO (`deque.popleft()`)

Explore niveau par niveau. Garantit le chemin le plus court, mais consomme beaucoup de mémoire.

DLS**Depth-Limited Search**

✗ Non optimal

Structure : DFS avec limite L

Variante du DFS bornée à L niveaux. Si aucune solution, L est incrémenté automatiquement (L initial = 3).

A***Recherche heuristique**

✓ Optimal

Structure : Tas min (heapq) – $f = g + h$

Combine coût réel $g(n)$ et heuristique $h(n)$. Optimal et complet avec une heuristique admissible.

Heuristique utilisée dans A*

$$f(n) = g(n) + h(n)$$

$g(n)$

Coût réel

Nombre de mouvements depuis l'état initial. Augmente de 1 à chaque mouvement.

$h(n)$

Coût estimé (heuristique)

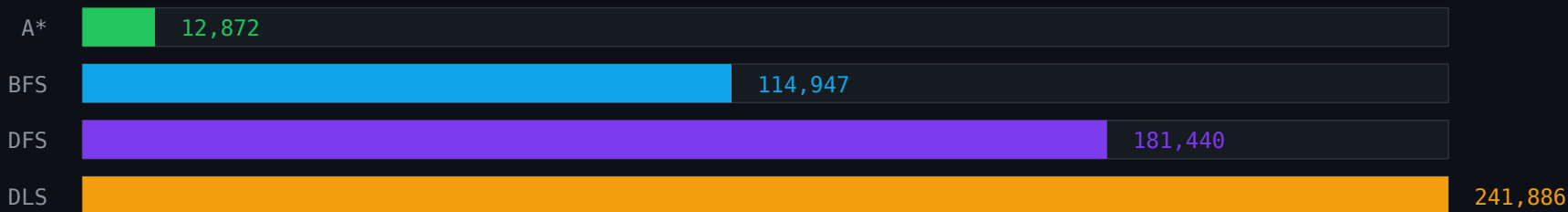
Nombre de pièces mal placées par rapport à l'état final. Admissible car ne surestime jamais le coût réel.

```
def h_mal_places(etat):  
    return sum(1 for i, v in enumerate(etat)  
               if v != 0 and v != ETAT_FINAL[i]) # ignore la case vide
```

Résultats expérimentaux

Algorithme	Nœuds générés	Nœuds visités	Temps (s)	Mémoire (KB)	Solution	Optimal
DFS	~181 440	~181 440	~2.0	~18 000	non optimal	✗
BFS	136 178	114 947	1.38	24 295	23 mvts	✓
DLS (L=30)	242 574	241 886	2.30	5 676	29 mvts	✗
A*	19 339	12 872	0.26	4 018	23 mvts	✓

Nœuds visités — comparaison visuelle



A* est de loin le plus performant : il explore **~14× moins de nœuds** que BFS et est **~5× plus rapide**, tout en garantissant la solution optimale. La clé : l'heuristique guide la recherche.

1er A*

Optimal + rapide + mémoire faible. Heuristique admissible.

2e BFS

Optimal mais mémoire élevée. Explore toute la frontière.

3e DFS

Peu de mémoire, mais non optimal. Peut explorer tout l'espace.

4e DLS

Améliore DFS en bornant L, mais surexplore si L trop grand.

Classement final : **A*** > **BFS** > **DFS** > **DLS**